with `@Transactional`.

Listing 10.25 Defining a saga transaction with database and event broker

```
@Service
public class OrderService {
  ...

  @Transactional                                                    ❶
  public Mono<Order> submitOrder(String isbn, int quantity) {
    return bookClient.getBookByIsbn(isbn)
      .map(book -> buildAcceptedOrder(book, quantity))
      .defaultIfEmpty(buildRejectedOrder(isbn, quantity))
      .flatMap(orderRepository::save)                               ❷
      .doOnNext(this::publishOrderAcceptedEvent);                   ❸
  }

  private void publishOrderAcceptedEvent(Order order) {
    if (!order.status().equals(OrderStatus.ACCEPTED)) {
      return;
    }
    var orderAcceptedMessage = new OrderAcceptedMessage(order.id());
    log.info("Sending order accepted event with id: {}", order.id());
    var result = streamBridge.send("acceptOrder-out-0",
      orderAcceptedMessage);
    log.info("Result of sending data for order with id {}: {}",
      order.id(), result);
  }
}
```

❶ Executes the method in a local transaction

❷ Saves the order in the database

❸ Publishes an event if the order is accepted

Spring Boot comes preconfigured with transaction management functionality and can handle transactional operations involving relational databases (as you learned in chapter 5). However, the channel established with RabbitMQ for the message producer is not transactional by default. To make the event-publishing operation join the existing transaction, we need to enable RabbitMQ's transactional support for the message producer in the application.yml file.